LinearSkills Academy

Class 10 — Tuples & Sets

Zero to Advance

Part 1 — Tuples: The Basics

1. What is a Tuple?

A tuple is like a list, but it cannot be changed after it is created. It uses round brackets ( ) instead of square brackets.

```
colors = ("red", "green", "blue")
numbers = (10, 20, 30)
single = (5,)   # comma is required for a one-item tuple
```

2. Why Use a Tuple? (Immutability)

Immutable means the data cannot be added to, removed, or changed once it is created.

```
colors = ("red", "green", "blue")
colors[0] = "yellow"   # Error! Tuples cannot be modified
```

Use a tuple when data should stay fixed and safe from accidental changes — for example coordinates, days of the week, or settings that must not change.

3. Indexing & Slicing

Reading from a tuple works exactly like a list — you just cannot change it.

```
colors = ("red", "green", "blue", "yellow")

print(colors[0])     # red
print(colors[-1])    # yellow
print(colors[1:3])   # ('green', 'blue')
```

4. Tuple Methods

Since tuples cannot change, there are only two useful methods:

Method	Example	Description
`.count()`	`colors.count("red")`	Counts how many times a value appears
`.index()`	`colors.index("blue")`	Finds the position of a value

Part 2 — Tuples: Going Further

5. Packing and Unpacking

Packing means putting values into a tuple. Unpacking means taking them back out into separate variables.

```
# Packing
person = ("Ali", 20, "Karachi")
```

`.intersection()`	`a.intersection(b)`	Items common to both sets
`.difference()`	`a.difference(b)`	Items in a but not in b

```
# Unpacking
name, age, city = person
print(name)   # Ali
print(age)    # 20
print(city)   # Karachi
```

6. Why Tuples Are Useful in Real Code

A very common use is returning more than one value from a function — Python actually packs them into a tuple automatically.

7. Nested Tuples

A tuple can hold other tuples inside it, just like nested lists.

```
student = ("Ali", (85, 90, 78))
print(student[0])       # Ali
print(student[1][0])    # 85
```

8. Tuple vs List — When to Choose Which

Situation	Use
Data will never change (fixed settings, coordinates)	Tuple
Data needs to grow, shrink, or be edited	List
Data will be used as a dictionary key	Tuple (lists cannot be used as keys)
You want to signal to other programmers "this won't change"	Tuple

Part 3 — Sets: The Basics

9. What is a Set?

A set stores multiple values like a list, but with two big differences: no duplicates are allowed, and there is no fixed order. It uses curly brackets { }.

```
fruits = {"apple", "banana", "mango", "apple"}
print(fruits)   # {'apple', 'banana', 'mango'} -- duplicate removed automatically
```

10. Common Set Methods

Method	Example	Description
`.add()`	`fruits.add("kiwi")`	Adds one item to the set
`.remove()`	`fruits.remove("banana")`	Removes an item (error if missing)
`.discard()`	`fruits.discard("banana")`	Removes an item (no error if missing)
`.union()`	`a.union(b)`	Combines two sets
`.intersection()`	`a.intersection(b)`	Items common to both sets
`.difference()`	`a.difference(b)`	Items in a but not in b

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b))          # {1, 2, 3, 4, 5, 6}
print(a.intersection(b))   # {3, 4}
print(a.difference(b))     # {1, 2}
```

Part 4 — Sets: Going Further

11. More Set Operations

Method	Example	Description
`.symmetric_difference()`	`a.symmetric_difference(b)`	Items in a or b, but not in both
`.issubset()`	`a.issubset(b)`	True if every item in a is also in b
`.issuperset()`	`a.issuperset(b)`	True if a contains every item in b
`.clear()`	`a.clear()`	Removes all items from the set

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.symmetric_difference(b))   # {1, 2, 5, 6}
print({1, 2}.issubset(a))          # True
print(a.issuperset({1, 2}))        # True
```

12. Set Comprehension

Just like list comprehension, but it builds a set instead. Useful when you want unique results in one line.

```
squares = {i * i for i in range(6)}
print(squares)   # {0, 1, 4, 9, 16, 25}
```

13. Frozenset

A frozenset is a set that cannot be changed after creation — the "tuple version" of a set. Useful when you need a set that must stay fixed, such as a dictionary key.

```
fixed_set = frozenset([1, 2, 3])
fixed_set.add(4)   # Error! frozenset cannot be modified
```

14. When to Use Each — Full Picture

Data Type	Use When...
List	You need an ordered collection that can change
Tuple	You need fixed data that should never change
Set	You need unique values only, and order doesn't matter
Frozenset	You need a fixed set, e.g. to use as a dictionary key

15. Mini Demo

```
students = ("Ali", "Sara", "Bilal")   # fixed list of students
grades_a = {"Ali", "Sara"}
grades_b = {"Sara", "Bilal"}

print("Students:", students)
print("In both A and B:", grades_a.intersection(grades_b))
print("Only in A:", grades_a.difference(grades_b))
print("Either A or B, not both:", grades_a.symmetric_difference(grades_b))
```